DOCUMENTO TÉCNICO

Plan de implementación de animaciones GSAP ScrollTrigger

Landing Page — Proyecto Pacífico

Alcance	features/landing/components (7 componentes)
Fecha	27 de octubre de 2026
Stack	Next.js 16.3.1 · React 19.2.8 · TypeScript 5 · Tailwind CSS v4
Librerías	gsap (ScrollTrigger) + @gsap/react
Estado	Propuesta — pendiente de aprobación

Este documento describe, sin modificar el código existente, la estrategia completa para implementar la secuencia de animaciones solicitada: barrido de entrada de la imagen a pantalla completa, aparición escalonada del contenido y colapso de la imagen de 100svh a 30svh ligado al scroll.

1. Objetivo de la experiencia

La meta es una secuencia de tres actos encadenados, percibida por el usuario como una sola pieza continua:

1. Acto I - Barrido de entrada. Al cargar la página, la imagen de fondo que cubre el 100% de la pantalla se revela con un barrido vertical de arriba hacia abajo (wipe), en lugar de aparecer de golpe.
2. Acto II - Entrada de contenido. Terminado el barrido, el navbar, el titulo PACIFICO y el subtitulo entran escalonadamente (stagger) sobre la imagen ya revelada.
3. Acto III - Colapso por scroll. Al empezar a hacer scroll hacia abajo, el bloque de la imagen se reduce progresivamente desde 100svh hasta 30svh, ligado al scroll (scrub). Una vez completado ese colapso, la página continúa con su scroll natural y las secciones siguientes (Hero, Features, Reviews, FAQ, Contact) fluyen normalmente.

El principio rector: el usuario nunca debe sentir un "salto". La transicion entre el acto III y el scroll natural debe ser invisible, y eso se logra con `pin` + `pinSpacing` de ScrollTrigger, no con `position: fixed` manual.

---

2. Estado actual del código (diagnostico previo)

Antes de animar hay que entender lo que existe. Esto es lo que encontre en el directorio.

2.1 Inventario de componentes

Archivo	Líneas	Tipo actual	Rol en la animación
LandingPage.tsx	53	Server Component	Orquestador. Aquí vive el contenedor animado.
LandingNavbar.tsx	42	Server Component	Entra en el Acto II. Debe quedar sticky tras el colapso.
LandingHero.tsx	57	Server Component	Primera sección del scroll natural (Acto III en adelante).
LandingFeatures.tsx	69	Server Component	Reveal por sección al hacer scroll.
LandingReviews.tsx	90	Server Component	Reveal por sección con stagger de tarjetas.
LandingFaq.tsx	92	Client ("use client")	Ya es cliente por el useState del acordeón.
LandingContact.tsx	59	Server Component	Reveal final.

2.2 Hallazgo bloqueante: return muerto en LandingPage.tsx

`LandingPage.tsx` tiene **dos sentencias `return` consecutivas**. El primer `return` (líneas 14-37) renderiza unicamente la sección de la imagen a pantalla completa con el navbar y el titulo. El segundo `return` (líneas 39-51), que contiene `LandingHero`, `LandingFeatures`, `LandingReviews`, `LandingFaq` y `LandingContact`, **es código inalcanzable: nunca se ejecuta**.

> **ATENCIÓN**
> Consecuencia directa para este plan: hoy no existe contenido debajo de la imagen, por lo tanto no hay recorrido de scroll. El Acto III es físicamente imposible de implementar sin antes fusionar ambos returns en uno solo. Este es el paso 1 obligatorio de la implementación.

2.3 Otras observaciones relevantes

- El layout raiz usa `<main className="p-4">`: hay un padding de 1rem alrededor de todo. Por eso la sección usa `min-h-[calc(100svh-2rem)]` y no `100svh`. Ese cálculo debe conservarse en los valores de la animación o el pin quedará desfasado 2rem.
- La imagen se renderiza con `next/image` usando `fill` + `priority` + `sizes="100vw"`. `priority` es correcto y necesario: sin el, la imagen podría no estar decodificada cuando arranca la animación de entrada.
- Existe un overlay `<div className="absolute inset-0 -z-10 bg-black/20" />` sobre la imagen. Debe animarse junto con la imagen o quedará visualmente desincronizado.
- El navbar usa enlaces ancla (`#features`, `#reviews`, `#faq`, `#contacts`). ScrollTrigger con `pin` altera las posiciones reales de esos anclajes; hay que preverlo (sección 9.3).
- `framer-motion@13.1.1` ya esta instalado. Conviene decidir si convive con GSAP o si se retira de la landing (sección 10).

3. Dependencias a instalar

Solo se necesitan dos paquetes. GSAP es gratuito en su totalidad desde la versión 3.13 (los antiguos plugins "Club" se liberaron), por lo que ScrollTrigger se instala con el paquete base.

```bash
npm install gsap @gsap/react
```

- `gsap` - libreria base + plugin ScrollTrigger incluido en `gsap/ScrollTrigger`.
- `@gsap/react` - provee el hook `useGSAP()`, que envuelve las animaciones en un `gsap.context()` y las revierte automáticamente al desmontar el componente. Es la forma oficial y recomendada de usar GSAP en React 18/19, donde el Strict Mode monta los componentes dos veces en desarrollo.

> **NOTA**
> No se requiere ninguna configuración en next.config.ts. GSAP se distribuye con builds ESM y funciona con el bundler de Next 16 sin transpilación adicional.

4. Arquitectura propuesta

4.1 Decisión clave: dónde vive el "use client"

GSAP manipula el DOM, así que necesita ejecutarse en el cliente. Hay dos caminos:

Opción	Cómo funciona	Veredicto
A) Marcar LandingPage como client	Agregar "use client" al orquestador	Rechazada. Convierte todo el árbol en cliente y sacrifica el beneficio de RSC en Features, Reviews y Contact, que son puramente estáticos.
B) Wrapper cliente con children	Un componente cliente nuevo que recibe los server components como children	Recomendada. Los componentes hijos se siguen renderizando en el servidor y llegan al wrapper como un árbol ya serializado. El wrapper solo añade refs y animación.

4.2 Archivos nuevos a crear

Todos dentro de `features/landing/`:

Archivo	Responsabilidad
components/LandingAnimator.tsx	Componente cliente. Registra los plugins, define el contenedor con ref y ejecuta el useGSAP maestro con los tres actos.
components/LandingHeroStage.tsx	Opcional pero recomendado. Aisla la "escena" de la imagen a pantalla completa (imagen + overlay + navbar + titulo) para que el Animator maneje refs limpias.
hooks/useLandingTimeline.ts	Opcional. Extrae la lógica de la línea de tiempo del componente para mantener el JSX legible y facilitar pruebas.

Archivo	Responsabilidad
lib/gsap.ts	Punto único de registro: gsap.registerPlugin(useGSAP, ScrollTrigger). Evita registros duplicados y centraliza la configuración global (gsap.defaults).

4.3 Estructura de marcado objetivo

La estructura del DOM es más importante que el código de animación. ScrollTrigger con `pin` inserta un elemento espaciador (pin-spacer) alrededor del elemento fijado, así que el árbol debe estar preparado.

```tsx
<main> {/* padding p-4, sin overflow-hidden */}
  <section ref={stageRef}>        {/* Elemento a PINEAR. No se anima directamente. */}
    <div ref={frameRef}>          {/* Marco que se encoge de 100svh a 30svh */}
      <div ref={imageWrapRef}>    {/* Capa con clip-path para el barrido */}
        <Image fill priority />
        <div ref={overlayRef} />  {/* El velo bg-black/20 */}
      </div>
      <div ref={navRef}>   <LandingNavbar /> </div>
      <div ref={titleRef}> <h1>PACIFICO</h1> <p>A simple way.</p> </div>
    </div>
  </section>

  <div ref={contentRef}>          {/* Secciones del scroll natural */}
    <LandingHero /> <LandingFeatures /> <LandingReviews />
    <LandingFaq /> <LandingContact />
  </div>
</main>
```

ATENCIÓN

Regla de oro de ScrollTrigger: nunca se anima el mismo elemento que se pinea. Por eso existen dos capas: stageRef se pinea y frameRef es el que cambia de altura. Mezclarlos produce jitter y cálculos de posición incorrectos.

5. Acto I - Barrido de entrada de la imagen

5.1 Técnica elegida: clip-path inset animado

Hay tres formas de hacer un barrido vertical. La comparacion:

Técnica	Rendimiento	Veredicto
Animar height del contenedor	Malo. Provoca layout/reflow en cada frame.	Descartada.
Máscara con un div que se desliza (translateY)	Bueno, usa solo transform.	Viable, pero exige un elemento extra y el fondo debe coincidir con el del body.
clip-path: inset(0 0 100% 0) -> inset(0 0 0% 0)	Bueno. Se compone en GPU en navegadores modernos.	Elegida. Una sola propiedad, sin DOM extra, y el efecto es exactamente un barrido de arriba hacia abajo.

El valor inicial `inset(0 0 100% 0)` recorta el 100% desde abajo, dejando la imagen invisible. Al llevarlo a `inset(0 0 0% 0)` el borde inferior del recorte baja hasta el pie del contenedor: la imagen se "dibuja" de arriba hacia abajo.

5.2 Prevención del FOUC (flash de contenido sin estilo)

Si el estado inicial se define solo en el JavaScript de GSAP, existe una ventana de uno o dos frames entre el pintado del HTML y la ejecución del hook donde la imagen aparece completa. Es un parpadeo visible.

- **Solución:** definir el estado inicial en CSS con una clase de Tailwind arbitraria, por ejemplo `[clip-path:inset(0_0_100%_0)]` en `imageWrapRef`, y que GSAP solo anime **hacia** el estado final. Así el servidor ya entrega el elemento recortado.
- Complementariamente, aplicar `gsap.set()` dentro de `useGSAP` antes de la línea de tiempo refuerza el estado por si el CSS cambia.
- Para el titulo y el navbar conviene `autoAlpha` (alias de GSAP que combina `opacity` + `visibility`), de modo que no sean clicables mientras estan invisibles.

5.3 Parámetros sugeridos

Propiedad	Valor	Justificación
duration	1.1 - 1.4 s	Por debajo de 0.9s el barrido se percibe como un corte; por encima de 1.6s el usuario siente espera.
ease	"power3.inOut" o "expo.out"	Arranque y frenado suaves. expo.out da una sensación más "premium" con salida larga.
scale de la imagen	de 1.12 a 1	Un ligero zoom-out simultáneo (efecto Ken Burns inverso) añade profundidad y disimula el borde del recorte.
delay inicial	0 - 0.15 s	Mínimo. El barrido debe sentirse como parte de la carga, no como una animación que llega tarde.

5.4 Esqueleto de la línea de tiempo del Acto I + II

```tsx
const intro = gsap.timeline({
  defaults: { ease: "power3.out" },
  onComplete: () => ScrollTrigger.refresh(),
});

intro
  .set(stageRef.current, { autoAlpha: 1 })
  .fromTo(imageWrapRef.current,
    { clipPath: "inset(0 0 100% 0)" },
    { clipPath: "inset(0 0 0% 0)", duration: 1.2, ease: "power3.inOut" })
  .from(imageRef.current,
    { scale: 1.12, duration: 1.6, ease: "expo.out" }, 0)   // paralelo, posicion 0
  .from(navRef.current,
    { autoAlpha: 0, y: -16, duration: 0.6 }, "-=0.45")
  .from(titleRef.current.children,
    { autoAlpha: 0, y: 24, duration: 0.7, stagger: 0.12 }, "-=0.35");
```

- Los parámetros de posición negativos (`"-=0.45"`) solapan los segmentos: el navbar empieza a entrar cuando al barrido le falta medio segundo. El solape es lo que convierte dos animaciones en una sola secuencia fluida.

- `titleRef.current.children` anima el `<h1>` y el `<p>` por separado con `stagger: 0.12`, dando el escalonado pedido.
- `ScrollTrigger.refresh()` en `onComplete` es importante: la línea de tiempo cambia el layout y los triggers deben recalcular sus posiciones.

6. Acto II - Entrada escalonada de los demás componentes

Hay que distinguir dos cosas que el requerimiento agrupa como "los demás componentes":

6.1 Contenido dentro de la pantalla inicial (navbar + titulo)

Ya resuelto en la línea de tiempo del punto 5.4. Son parte de la `intro`, no llevan ScrollTrigger: ocurren por tiempo, inmediatamente después del barrido.

6.2 Secciones debajo del pliegue (Hero, Features, Reviews, FAQ, Contact)

Estas **no** deben animarse por tiempo. Deben animarse cuando el usuario llega a ellas, con un ScrollTrigger por sección y `toggleActions`, no `scrub`.

```tsx
gsap.utils.toArray<HTMLElement>("[data-reveal]").forEach((section) => {
  gsap.from(section, {
    autoAlpha: 0,
    y: 48,
    duration: 0.8,
    ease: "power2.out",
    scrollTrigger: {
      trigger: section,
      start: "top 85%",      // dispara cuando el borde superior cruza el 85% del viewport
      toggleActions: "play none none reverse",
    },
  });
});
```

- Se marca cada `<section>` con `data-reveal` en los componentes existentes. Es un atributo, no un cambio estructural: una sola línea por archivo.
- Para las rejillas de tarjetas (Features con 4 tarjetas, Reviews con 3) conviene un segundo tween con `stagger: 0.1` sobre los hijos, para que las tarjetas no entren como un bloque.
- `toggleActions: "play none none reverse"` significa: al entrar reproduce, al salir por abajo no hace nada, al volver a entrar nada, y al salir por arriba revierte. Es el comportamiento natural de un reveal reversible.

> **ATENCIÓN**
> No combinar scrub y toggleActions en el mismo trigger. Son modelos mutuamente excluyentes: scrub ata el progreso al scroll, toggleActions dispara reproducciones discretas.

7. Acto III - Colapso de la imagen de 100svh a 30svh

Este es el corazón técnico del requerimiento y donde se concentran los errores típicos.

7.1 El mecanismo: pin + scrub

La idea: cuando el borde superior de la sección toca el borde superior del viewport, se **pinea** la sección durante una distancia de scroll determinada. Durante esa distancia, el usuario hace scroll pero la sección no se mueve; en su lugar, su progreso (de 0 a 1) controla la altura del marco interno. Al agotarse la distancia, ScrollTrigger despinea y, gracias al espaciador que inserta automáticamente (`pinSpacing: true`, el valor por defecto), la página continúa su scroll natural exactamente donde corresponde, sin salto.

```tsx
const collapse = gsap.timeline({
  scrollTrigger: {
    trigger: stageRef.current,
    start: "top top",
    end: "+=60%",        // distancia de colapso: 60% de la altura del viewport
    scrub: 0.8,          // suavizado: la animacion "persigue" al scroll con 0.8s de inercia
    pin: true,
    pinSpacing: true,
    anticipatePin: 1,
    invalidateOnRefresh: true,
    id: "hero-collapse",
  },
});

collapse
  .to(frameRef.current, {
    height: () => window.innerHeight * 0.3,
    ease: "none",
  }, 0)
  .to(titleRef.current, { autoAlpha: 0, y: -24, ease: "none" }, 0)
  .to(overlayRef.current, { opacity: 0.45, ease: "none" }, 0);
```

7.2 Decisiones de parámetros, explicadas

Parámetro	Valor propuesto	Por que
start	"top top"	El colapso debe empezar exactamente cuando la sección ocupa el viewport completo. Es también el valor por defecto cuando pin: true.
end	"+=60%"	Define cuánto scroll "cuesta" el colapso. Con 60% de la altura del viewport el gesto se siente deliberado pero no tedioso. Valores por encima de 100% hacen que el usuario sienta que la página esta trabada.
scrub	0.8	Un número (en lugar de true) introduce inercia: la animación alcanza al scroll en 0.8s. Elimina la sensación "pegada al pixel" y absorbe el scroll por saltos del trackpad y la rueda del ratón.
ease	"none" en los tweens	Obligatorio con scrub. Cualquier otra curva desincroniza la posición visual respecto al dedo/rueda del usuario.
anticipatePin	1	Aplica el pin un frame antes. Evita el micro-salto en scroll rápido, especialmente en Safari.

Parámetro	Valor propuesto	Por que
invalidateOnRefresh	true	Fuerza a recalcular los valores de funcion (como window.innerHeight * 0.3) en cada refresh. Sin esto, girar el teléfono deja la altura calculada con el viewport viejo.

7.3 Por qué animar height aquí es aceptable (y cómo mitigarlo)

En el Acto I se descarto animar `height` por costoso. Aquí es distinto y la excepción es deliberada:

- El requerimiento es explícito: la imagen debe **reducirse a 30% de height**, dejando espacio real para el contenido siguiente. Un `scaleY` no libera espacio en el flujo del documento; solo deformaría la imagen.
- Durante el pin, el contenido inferior esta fuera del viewport: el reflow afecta a un subárbol pequeño.
- Mitigación recomendada: aplicar `will-change: height` mientras el trigger esta activo (via callbacks `onToggle`) y retirarlo después. Mantener `will-change` permanente consume memoria de GPU innecesariamente.
- Alternativa si se detectan caidas de FPS en dispositivos de gama baja: animar la propiedad CSS personalizada `--stage-h` con `gsap.to(el, { "--stage-h": "30svh" })` y que el CSS consuma `height: var(--stage-h)`. El costo es el mismo, pero centraliza el valor y evita que GSAP escriba estilos en línea.

7.4 Unidades: svh, no vh

El código actual ya usa `min-h-[calc(100svh-2rem)]` — una decisión correcta que debe mantenerse. En móviles, `100vh` incluye la zona que ocupa la barra de direcciones, lo que provoca un salto cuando esa barra se oculta al hacer scroll. `svh` (small viewport height) usa la altura con la UI del navegador visible y permanece estable. Combinado con ScrollTrigger esto es crítico: un cambio de altura del viewport a mitad de scroll dispara un refresh y puede reposicionar el pin.

7.5 La transicion al scroll natural

No requiere código. Es consecuencia de `pinSpacing: true`: ScrollTrigger inserta un contenedor con la altura exacta de la distancia pineada. Al llegar al `end`, la sección se despinea y el documento prosigue. El único cuidado es **no poner `overflow: hidden` en `<main>` ni en ningún ancestro**, porque eso rompe el pinning.

Para que el remate sea pulido, conviene que al terminar el colapso el navbar quede fijo arriba. Dos opciones: dejar el navbar fuera del `frameRef` con `position: sticky; top: 0`, o animarlo a una barra compacta con fondo dentro de la misma línea de tiempo del colapso. La segunda opción es más vistosa y usa la misma infraestructura.

8. Accesibilidad y responsive con gsap.matchMedia()

Esta sección no es opcional. Una animación de pin a pantalla completa es exactamente el tipo de efecto que causa malestar a personas con trastornos vestibulares.

```tsx
const mm = gsap.matchMedia();

mm.add({
  isDesktop: "(min-width: 768px)",
  isMobile: "(max-width: 767px)",
  reduceMotion: "(prefers-reduced-motion: reduce)",
}, (ctx) => {
  const { isDesktop, reduceMotion } = ctx.conditions as Record<string, boolean>;

  if (reduceMotion) {
    gsap.set([imageWrapRef.current, navRef.current, titleRef.current], {
      clipPath: "inset(0 0 0% 0)", autoAlpha: 1, y: 0,
    });
    return;   // sin pin, sin scrub: la pagina se comporta como una landing estatica
  }

  // ... lineas de tiempo de los Actos I, II y III
  // la distancia de colapso se ajusta por dispositivo:
  //   end: isDesktop ? "+=60%" : "+=45%"
}, containerRef);
```

- `gsap.matchMedia()` revierte automáticamente todo lo creado dentro de un bloque cuando su condición deja de cumplirse. Girar una tablet de vertical a horizontal limpia y reconstruye las animaciones sin trabajo manual.
- **No anidar `gsap.context()` dentro de `matchMedia`**: matchMedia ya crea un contexto internamente. Con `useGSAP` basta pasar el `scope`.
- En móvil conviene acortar la distancia del colapso (45% en lugar de 60%): el scroll táctil recorre más rápido y un pin largo se siente como que la página se trabó.
- Con `prefers-reduced-motion` se saltan por completo el pin y el scrub, no solo se acortan las duraciones. El pin es el elemento que causa desorientación.

---

9. Riesgos conocidos y cómo mitigarlos

9.1 Imagenes de next/image y el cálculo de posiciones

`next/image` con `fill` carga de forma asíncrona. Si ScrollTrigger calcula las posiciones antes de que la imagen termine de decodificarse, las distancias de los triggers quedan mal.

- Mitigación: llamar a `ScrollTrigger.refresh()` en el `onLoad` de la imagen principal, o un `ScrollTrigger.refresh()` global tras `window.load`.
- El atributo `priority` ya presente en el código reduce mucho el riesgo para la imagen del hero, pero las imagenes de `LandingHero` (dos `next/image` con `fill`) no lo tienen y si afectan las posiciones de los triggers posteriores.

9.2 React 19 Strict Mode y doble montaje

En desarrollo, React monta y desmonta cada componente dos veces. Sin limpieza, se crean ScrollTriggers duplicados que pelean entre si (síntoma típico: la animación va al doble de velocidad o parpadea).

- Mitigación: usar `useGSAP()` exclusivamente. El hook crea un `gsap.context()` y llama a `revert()` en la limpieza, eliminando tweens y triggers. No crear animaciones en `useEffect` sin contexto.
- Nunca crear animaciones GSAP en el cuerpo del componente ni en manejadores de evento sin envolverlos con `contextSafe()`.

9.3 Enlaces ancla del navbar bajo un pin

Los `href="#features"` del navbar usan el scroll nativo del navegador, que desconoce el espaciador del pin. El resultado habitual es aterrizar desplazado.

- Mitigación A (mínima): añadir `scroll-behavior: smooth` y `scroll-margin-top` a las secciones destino.
- Mitigación B (robusta): interceptar el clic y usar `gsap.to(window, { scrollTo: ... })` con el plugin ScrollToPlugin, que respeta las posiciones reales. Requiere registrar un plugin adicional.

9.4 Interferencia con framer-motion

`framer-motion@13.1.1` esta en las dependencias. Si alguna sección de la landing ya usa `motion` con animaciones de scroll (`useScroll`, `whileInView`), convivira con ScrollTrigger sobre los mismos elementos y los resultados serán impredecibles.

- Mitigación: auditar el uso de framer-motion en la landing antes de empezar. Regla: una sola libreria de animación por elemento. Lo más limpio es que GSAP gobierne toda la landing y framer-motion quede para el resto de la app.

9.5 Layout shift en la primera carga (CLS)

Si el estado inicial del titulo es `autoAlpha: 0` aplicado por JS, hay un instante en que ocupa espacio visible. Con `autoAlpha` (que añade `visibility: hidden`) el elemento conserva su caja, así que no hay desplazamiento; usar `display: none` si lo causaria. Es otra razon para preferir `autoAlpha` sobre `opacity`.

9.6 Markers en produccion

`markers: true` es indispensable durante el desarrollo para ver dónde empieza y termina cada trigger. Es igual de indispensable eliminarlo antes de hacer merge.

- Patron sugerido: `markers: process.env.NODE_ENV === "development"`. Se autolimpia y evita el olvido.

---

10. Plan de ejecución por fases

Fase 0 - Preparación (bloqueante)

1. Fusionar los dos `return` de `LandingPage.tsx` en uno solo, de modo que las cinco secciones inferiores se rendericen realmente. **Sin este paso no hay scroll y nada del Acto III es verificable.**
2. Verificar visualmente que la página completa se ve correcta en estático, sin ninguna animación todavia.
3. Instalar `gsap` y `@gsap/react`.
4. Crear `features/landing/lib/gsap.ts` con el registro de plugins.

Fase 1 - Estructura

1. Crear `LandingAnimator.tsx` con `"use client"` y el patron de `children`.
2. Reestructurar el marcado del hero según el esquema de la sección 4.3: capa a pinear separada de la capa que se encoge.
3. Añadir `data-reveal` a las cinco secciones inferiores.
4. Confirmar que ningún ancestro tiene `overflow: hidden`.

Fase 2 - Acto I y II

1. Definir el estado inicial del `clip-path` en CSS para evitar el FOUC.
2. Construir la línea de tiempo `intro` con el barrido, el zoom-out y el stagger de navbar y titulo.
3. Ajustar duraciones y solapes hasta que la secuencia se perciba como una sola pieza.

Fase 3 - Acto III

1. Crear el ScrollTrigger con `pin`, `scrub` y `end: "+=60%"`, con `markers: true`.
2. Calibrar la distancia del `end` probando con rueda, trackpad y táctil.
3. Añadir las animaciones secundarias del colapso: desvanecido del titulo y ajuste del overlay.
4. Verificar que la salida del pin hacia el scroll natural no produzca ningún salto.

Fase 4 - Reveals de secciones

1. Implementar el bucle sobre `[data-reveal]` con `toggleActions`.
2. Añadir stagger a las tarjetas de Features y Reviews.
3. Revisar que el reveal no entre en conflicto con el acordeón de `LandingFaq`, que cambia de altura al abrirse (requiere `ScrollTrigger.refresh()` al togglear).

Fase 5 - Robustez

1. Envolver todo en `gsap.matchMedia()` con las ramas desktop, móvil y `prefers-reduced-motion`.
2. Añadir los `ScrollTrigger.refresh()` en los puntos de carga de imagen.
3. Resolver los enlaces ancla del navbar.
4. Retirar `markers` o condicionarlos al entorno.

Fase 6 - Verificación

1. Pruebas en Chrome, Safari y Firefox de escritorio, y en Safari iOS y Chrome Android.
2. Medir FPS durante el colapso con el panel Performance de las DevTools. Objetivo: mantenerse en 60fps, sin frames largos por encima de 50ms.
3. Verificar con `prefers-reduced-motion: reduce` activado en el sistema operativo.
4. Verificar recarga con el scroll ya avanzado a media página: el navegador restaura la posición y ScrollTrigger debe reconstruir el estado correctamente.
5. Comprobar que no hay regresión de CLS ni de LCP respecto al estado previo.

11. Criterios de aceptación

#	Criterio	Cómo se verifica
1	La imagen se revela con un barrido de arriba hacia abajo en la carga inicial	Inspección visual y grabación de pantalla a cámara lenta
2	El navbar y el titulo entran escalonados después del barrido, no al mismo tiempo	Inspección visual

#	Criterio	Cómo se verifica
3	Al hacer scroll hacia abajo, la altura del bloque pasa de 100svh a 30svh de forma proporcional al scroll	Scroll parcial: al soltar a mitad del recorrido, la altura debe quedar a mitad de camino
4	Al invertir el scroll, la animación se revierte proporcionalmente	Scroll hacia arriba a mitad del colapso
5	Tras completarse el colapso, el scroll continúa sin salto ni solapamiento	Scroll continuo de arriba a abajo sin pausas
6	Las secciones inferiores aparecen al entrar en el viewport	Scroll lento por toda la página
7	Con prefers-reduced-motion activado no hay pin ni scrub y todo el contenido es visible	Activar la preferencia en el SO y recargar
8	No hay ScrollTriggers duplicados en desarrollo	ScrollTrigger.getAll().length en la consola debe coincidir con el número esperado
9	No quedan markers en el build de produccion	Build local y revision visual
10	El rendimiento se mantiene en 60fps durante el colapso	Panel Performance de DevTools

12. Resumen ejecutivo

- El requerimiento se descompone en tres actos: barrido de entrada (clip-path por tiempo), entrada escalonada de contenido (stagger por tiempo) y colapso de 100svh a 30svh (pin + scrub ligado al scroll).
- Existe un bloqueante previo: el `return` muerto en `LandingPage.tsx` deja sin renderizar las cinco secciones inferiores, por lo que hoy no existe recorrido de scroll. Arreglarlo es la Fase 0.
- La arquitectura recomendada es un wrapper cliente con `children`, que preserva los Server Components de Next.js y aisla GSAP en una sola frontera de cliente.
- Las tres claves técnicas: nunca animar el elemento pineado, usar `ease: "none"` con `scrub`, y envolver todo en `gsap.matchMedia()` para responsive y accesibilidad.
- El esfuerzo estimado es de 6 fases; la mayor parte del tiempo se va en calibrar la distancia del colapso y en las pruebas entre navegadores, no en escribir el código de animación.